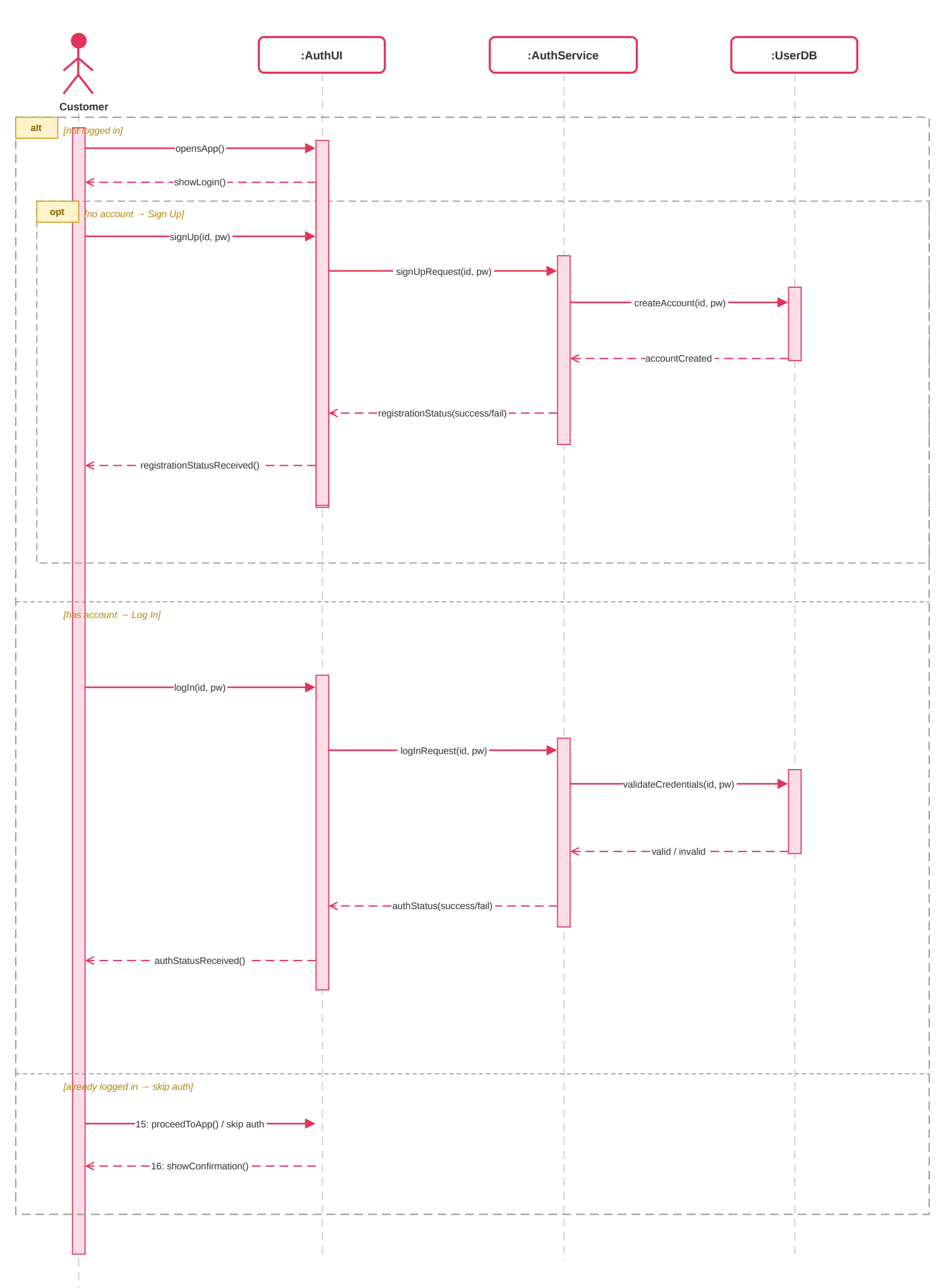
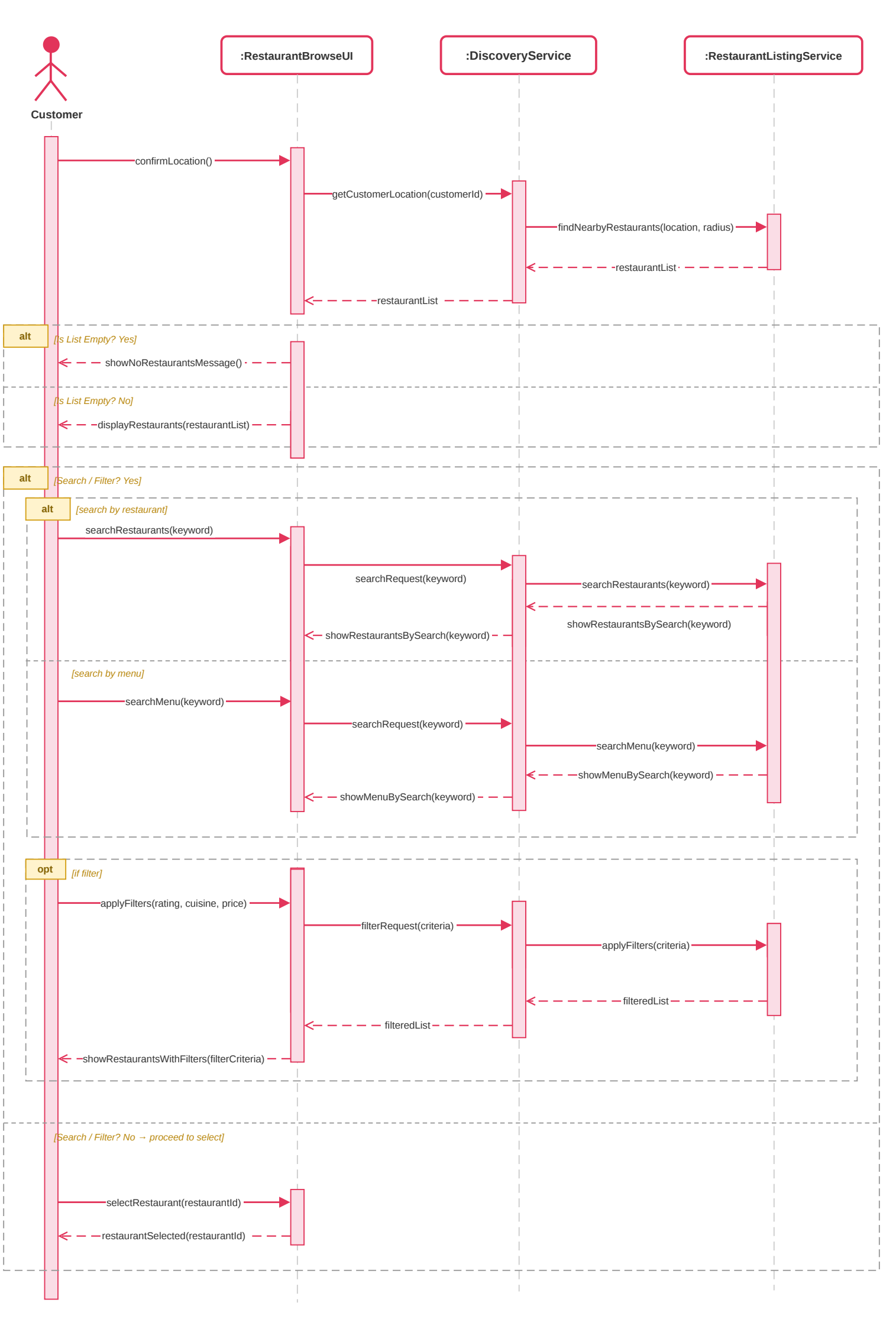
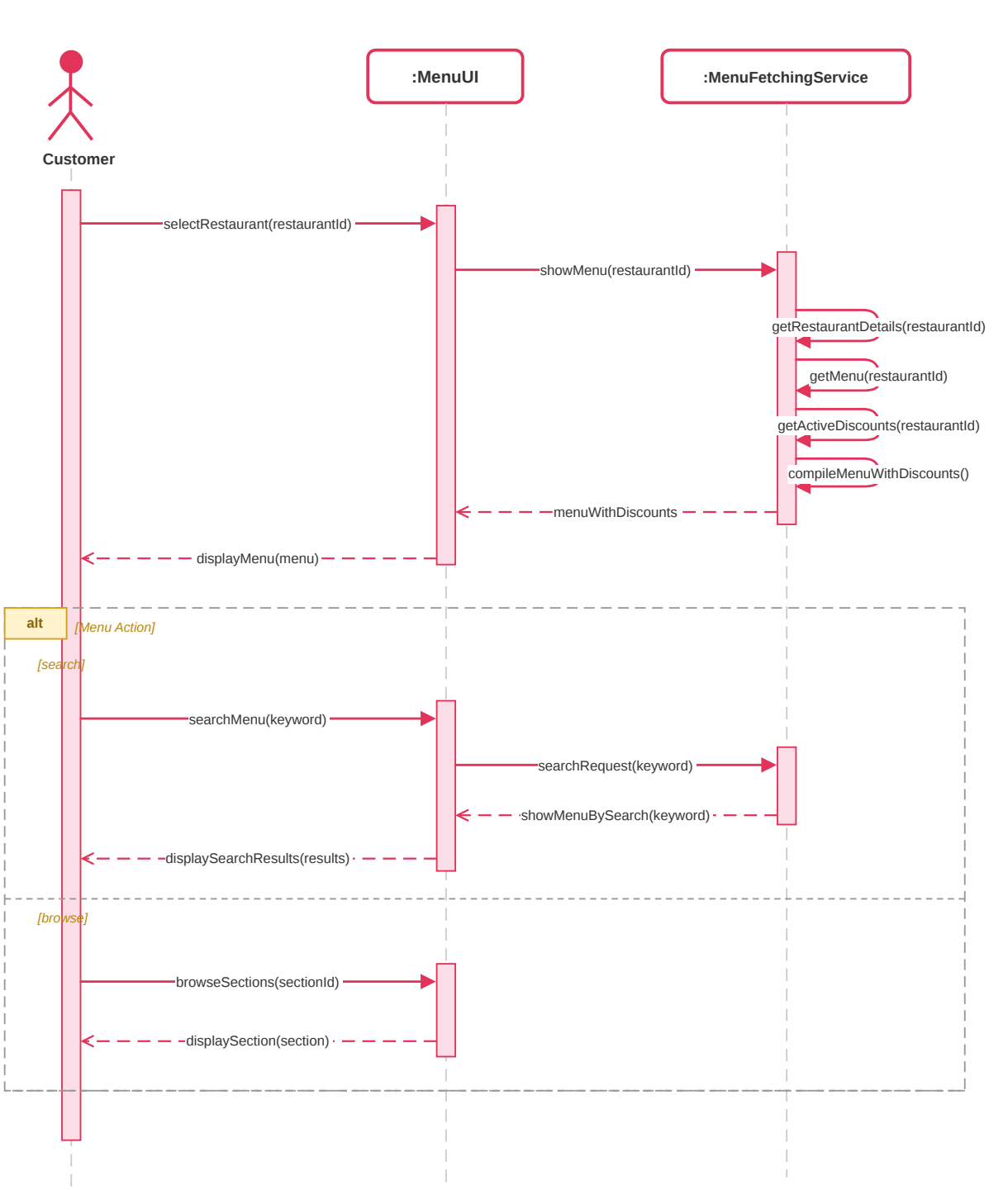
1. Authentication (Sign Up & Log In)


:AuthUI
:AuthService
:UserDB
Customer
alt
[not logged in]
opensApp()
showLogin()
opt
[no account → Sign Up]
signUp(id, pw)
signUpRequest(id, pw)
createAccount(id, pw)
accountCreated
registrationStatus(success/fail)
registrationStatusReceived()
[has account → Log In]
logIn(id, pw)
logInRequest(id, pw)
validateCredentials(id, pw)
valid / invalid
authStatus(success/fail)
authStatusReceived()
[already logged in → skip auth]
15: proceedToApp() / skip auth
16: showConfirmation()

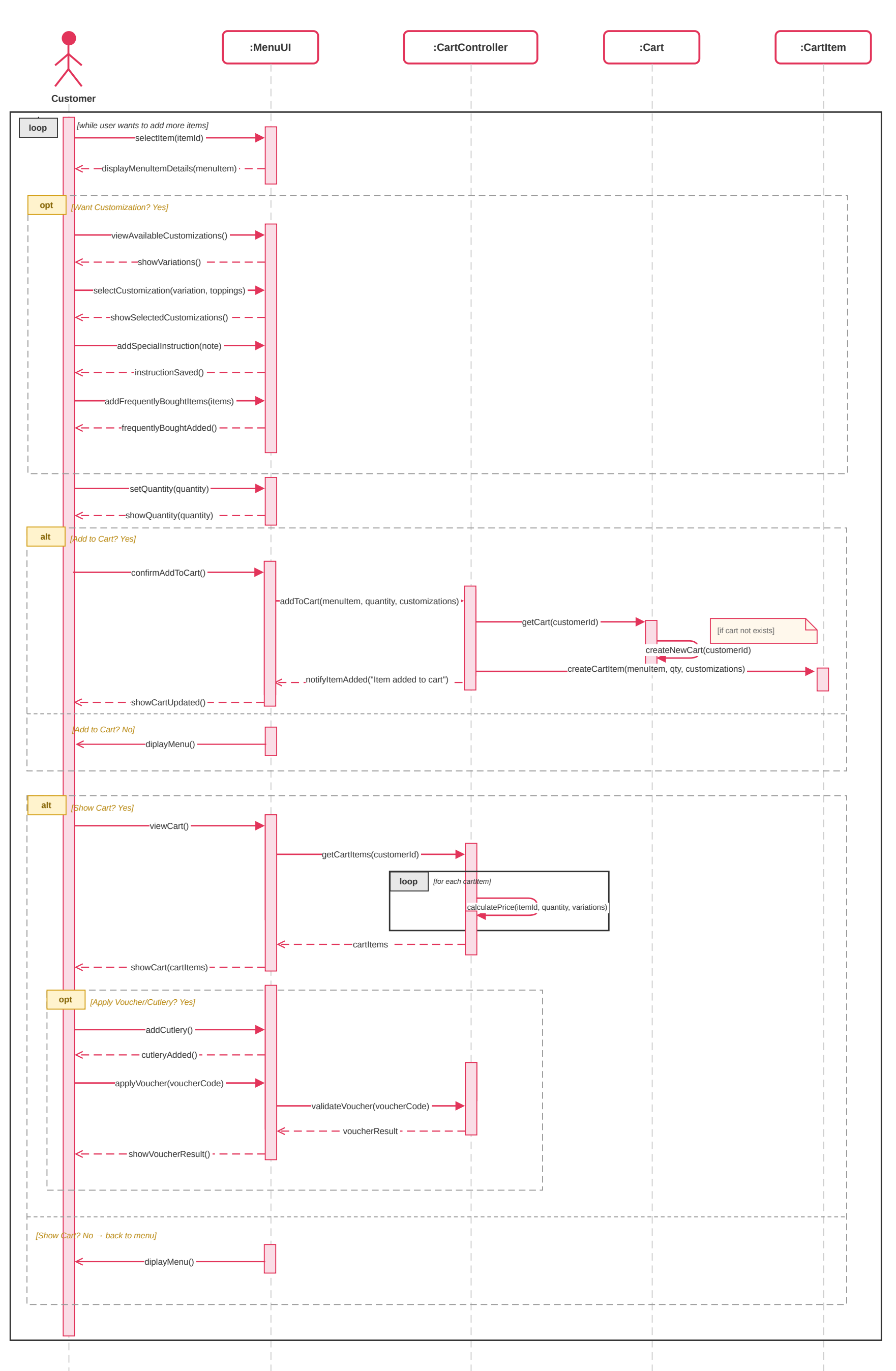
2. Restaurant Discovery & Search


:RestaurantBrowseUI
:DiscoveryService
:RestaurantListingService
Customer
confirmLocation()
getCustomerLocation(customerId)
findNearbyRestaurants(location, radius)
restaurantList
restaurantList
alt
[Is List Empty? Yes]
showNoRestaurantsMessage()
[Is List Empty? No]
displayRestaurants(restaurantList)
alt
[Search / Filter? Yes]
alt
[search by restaurant]
searchRestaurants(keyword)
searchRequest(keyword)
searchRestaurants(keyword)
showRestaurantsBySearch(keyword)
showRestaurantsBySearch(keyword)
[search by menu]
searchMenu(keyword)
searchRequest(keyword)
searchMenu(keyword)
showMenuBySearch(keyword)
showMenuBySearch(keyword)
opt
[If filter]
applyFilters(rating, cuisine, price)
filterRequest(criteria)
applyFilters(criteria)
filteredList
filteredList
showRestaurantsWithFilters(filterCriteria)
[Search / Filter? No → proceed to select]
selectRestaurant(restaurantId)
restaurantSelected(restaurantId)

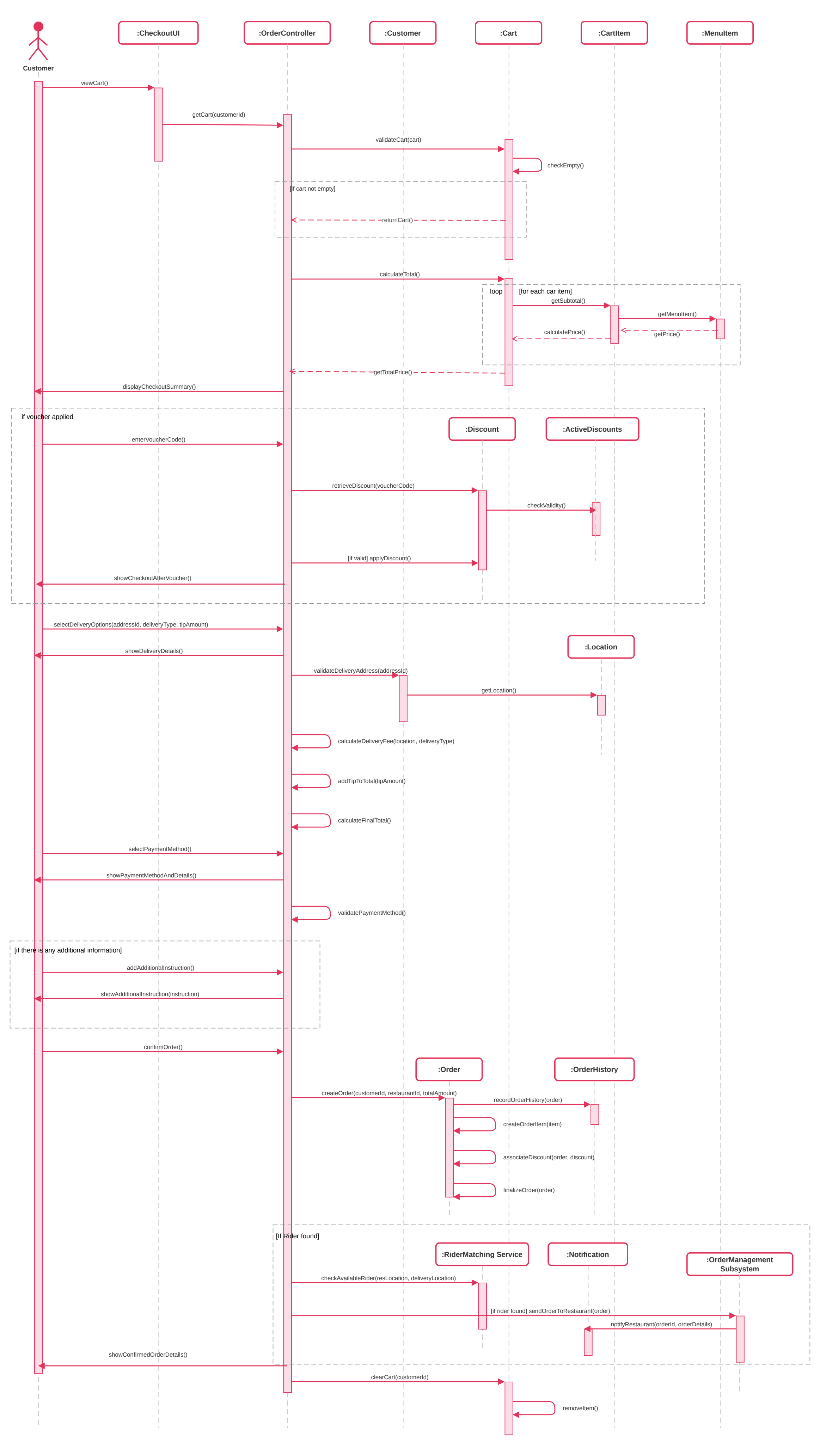
3. Menu Browsing


:MenuUI
:MenuFetchingService
Customer
selectRestaurant(restaurantId)
showMenu(restaurantId)
getRestaurantDetails(restaurantId)
getMenu(restaurantId)
getActiveDiscounts(restaurantId)
compileMenuWithDiscounts()
menuWithDiscounts
displayMenu(menu)
alt
[Menu Action]
[search]
searchMenu(keyword)
searchRequest(keyword)
showMenuBySearch(keyword)
displaySearchResults(results)
[browse]
browseSections(sectionId)
displaySection(section)

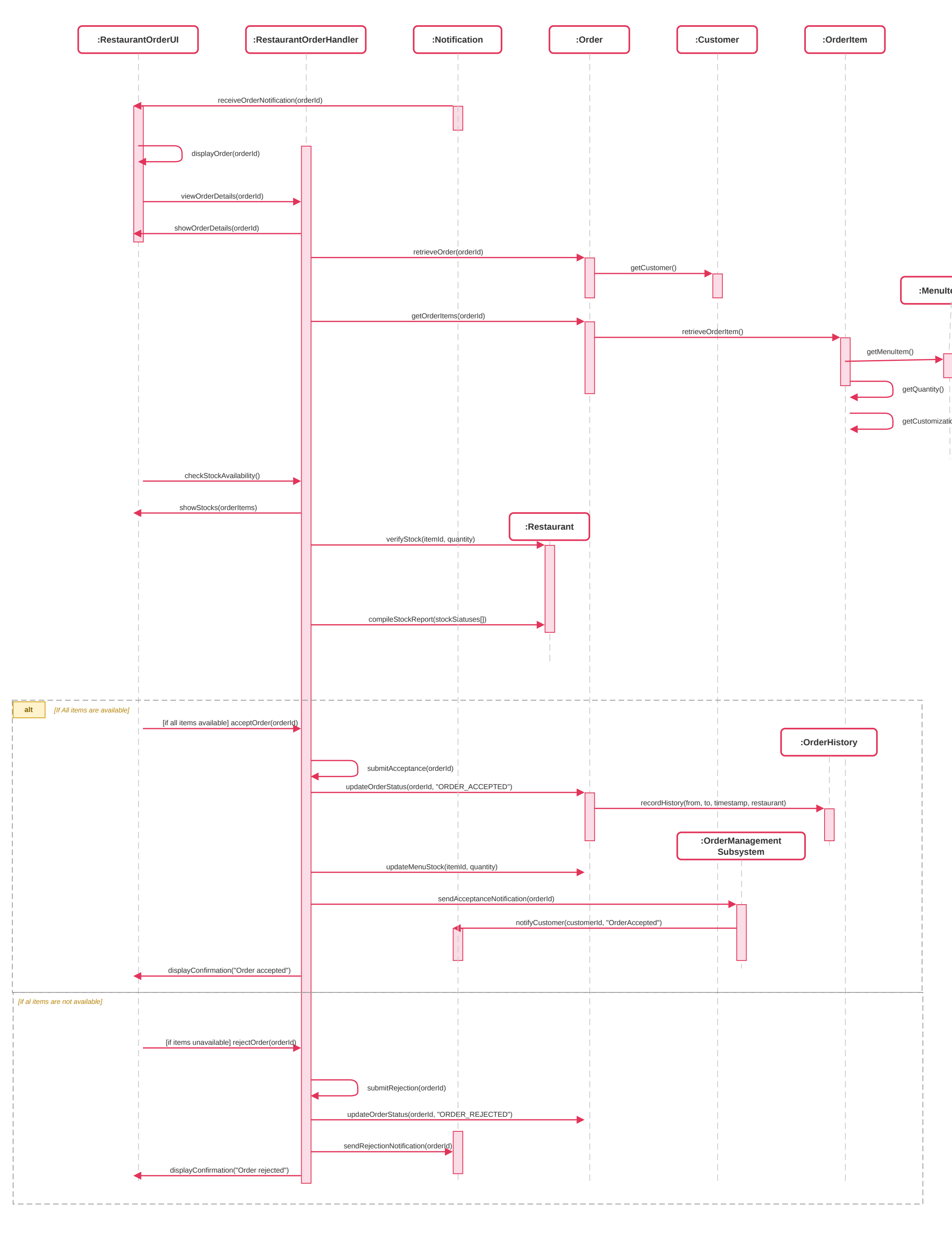
4. Cart & Customization


:MenuUI
:CartController
:Cart
:CartItem
Customer
loop
[while user wants to add more items]
selectItem(itemId)
displayMenuItemDetails(menuItem)
opt
[Want Customization? Yes]
viewAvailableCustomizations()
showVariations()
selectCustomization(variation, toppings)
showSelectedCustomizations()
addSpecialInstruction(note)
instructionSaved()
addFrequentlyBoughtItems(items)
frequentlyBoughtAdded()
setQuantity(quantity)
showQuantity(quantity)
alt
[Add to Cart? Yes]
confirmAddToCart()
addToCart(menuItem, quantity, customizations)
getCart(customerId)
[if cart not exists]
createNewCart(customerId)
createCartItem(menuItem, qty, customizations)
notifyItemAdded("Item added to cart")
showCartUpdated()
[Add to Cart? No]
diplayMenu()
alt
[Show Cart? Yes]
viewCart()
getCartItems(customerId)
loop
[for each cartItem]
calculatePrice(itemId, quantity, variations)
cartItems
showCart(cartItems)
opt
[Apply Voucher/Cutlery? Yes]
addCutlery()
cutleryAdded()
applyVoucher(voucherCode)
validateVoucher(voucherCode)
voucherResult
showVoucherResult()
[Show Cart? No → back to menu]
diplayMenu()

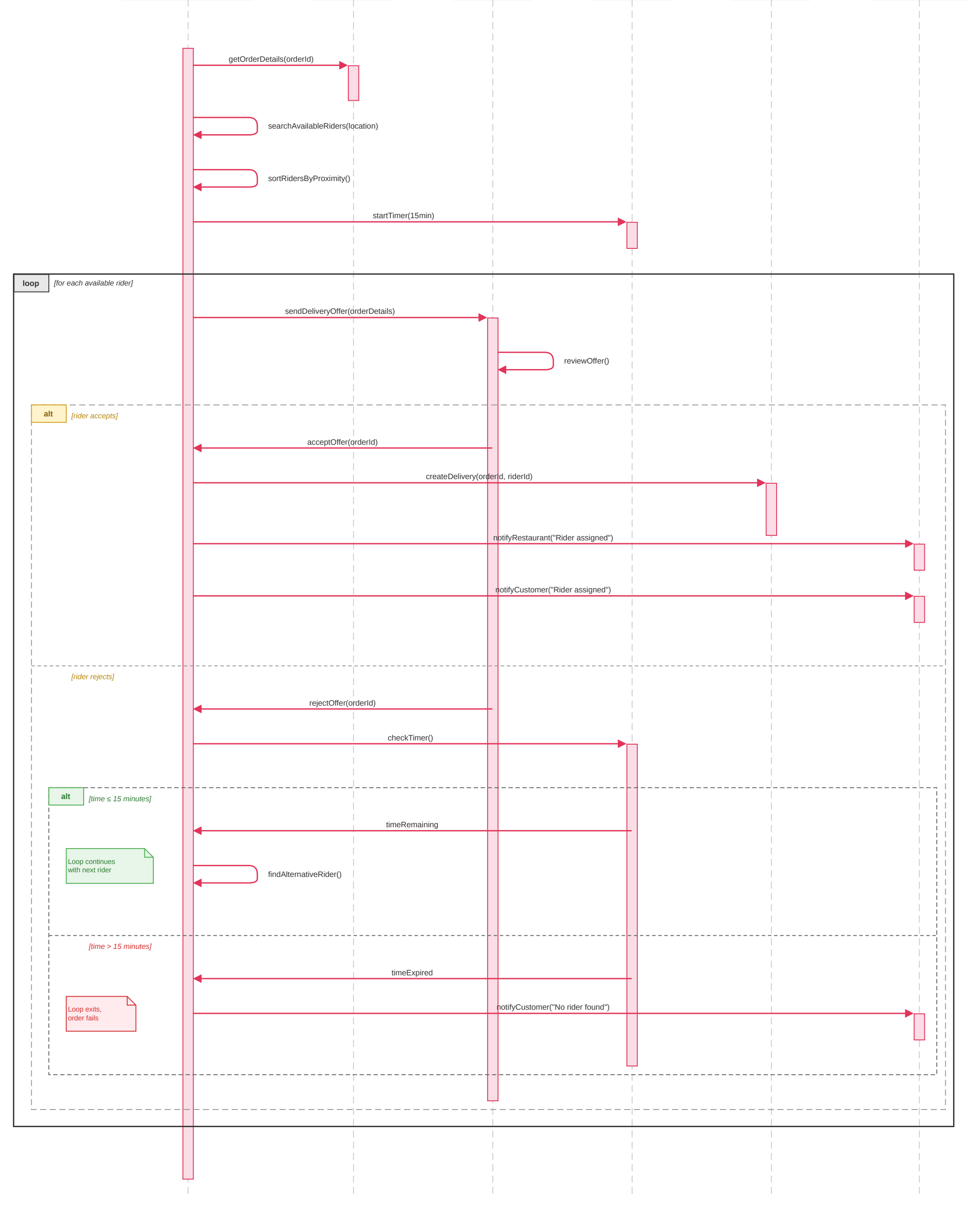
5. Checkout & Order Placement


:CheckoutUI
:OrderController
:Customer
:Cart
:CartItem
:MenuItem
Customer
viewCart()
getCart(customerId)
validateCart(cart)
checkEmpty()
[if cart not empty]
returnCart()
calculateTotal()
loop
[for each car item]
getSubtotal()
getMenuItem()
calculatePrice()
getPrice()
getTotalPrice()
displayCheckoutSummary()
if voucher applied
:Discount
:ActiveDiscounts
enterVoucherCode()
retrieveDiscount(voucherCode)
checkValidity()
[if valid] applyDiscount()
showCheckoutAfterVoucher()
selectDeliveryOptions(addressId, deliveryType, tipAmount)
showDeliveryDetails()
:Location
validateDeliveryAddress(addressId)
getLocation()
calculateDeliveryFee(location, deliveryType)
addTipToTotal(tipAmount)
calculateFinalTotal()
selectPaymentMethod()
showPaymentMethodAndDetails()
validatePaymentMethod()
[if there is any additional information]
addAdditionalInstruction()
showAdditionalInstruction(instruction)
confirmOrder()
:Order
:OrderHistory
createOrder(customerId, restaurantId, totalAmount)
recordOrderHistory(order)
createOrderItem(item)
associateDiscount(order, discount)
finalizeOrder(order)
[If Rider found]
:RiderMatching Service
:Notification
:OrderManagement Subsystem
checkAvailableRider(resLocation, deliveryLocation)
[if rider found] sendOrderToRestaurant(order)
notifyRestaurant(orderId, orderDetails)
showConfirmedOrderDetails()
clearCart(customerId)
removeItem()

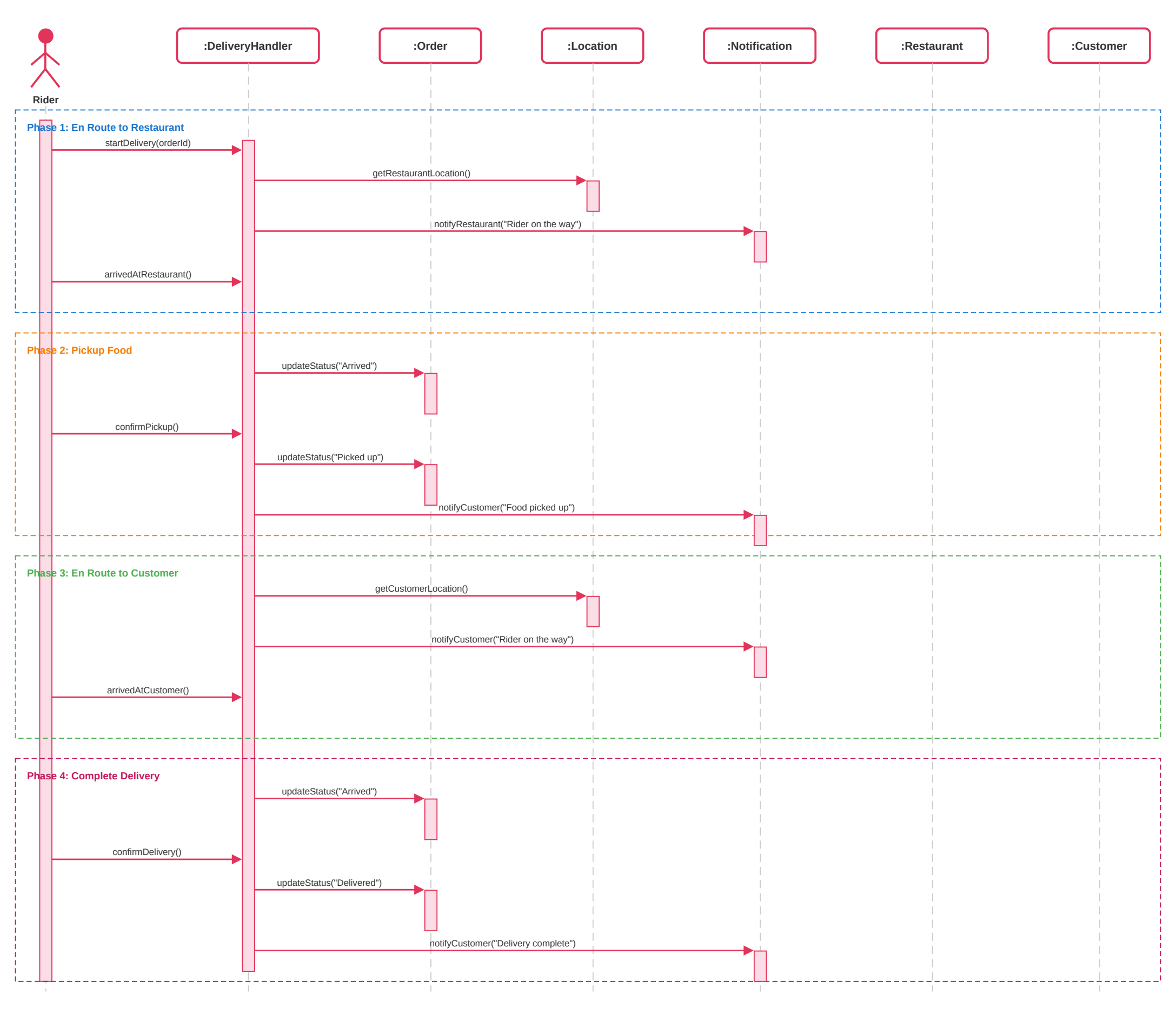
6. Order Processing – Restaurant


:RestaurantOrderUI
:RestaurantOrderHandler
:Notification
:Order
:Customer
:OrderItem
receiveOrderNotification(orderId)
displayOrder(orderId)
viewOrderDetails(orderId)
showOrderDetails(orderId)
retrieveOrder(orderId)
getCustomer()
:MenuItem
getOrderItems(orderId)
retrieveOrderItem()
getMenuItem()
getQuantity()
getCustomization()
checkStockAvailability()
showStocks(orderItems)
:Restaurant
verifyStock(itemId, quantity)
compileStockReport(stockStatuses[])
alt
[If All items are available]
[if all items available] acceptOrder(orderId)
:OrderHistory
submitAcceptance(orderId)
updateOrderStatus(orderId, "ORDER_ACCEPTED")
recordHistory(from, to, timestamp, restaurant)
:OrderManagement Subsystem
updateMenuStock(itemId, quantity)
sendAcceptanceNotification(orderId)
notifyCustomer(customerId, "OrderAccepted")
displayConfirmation("Order accepted")
[if al items are not available]
[if items unavailable] rejectOrder(orderId)
submitRejection(orderId)
updateOrderStatus(orderId, "ORDER_REJECTED")
sendRejectionNotification(orderId)
displayConfirmation("Order rejected")

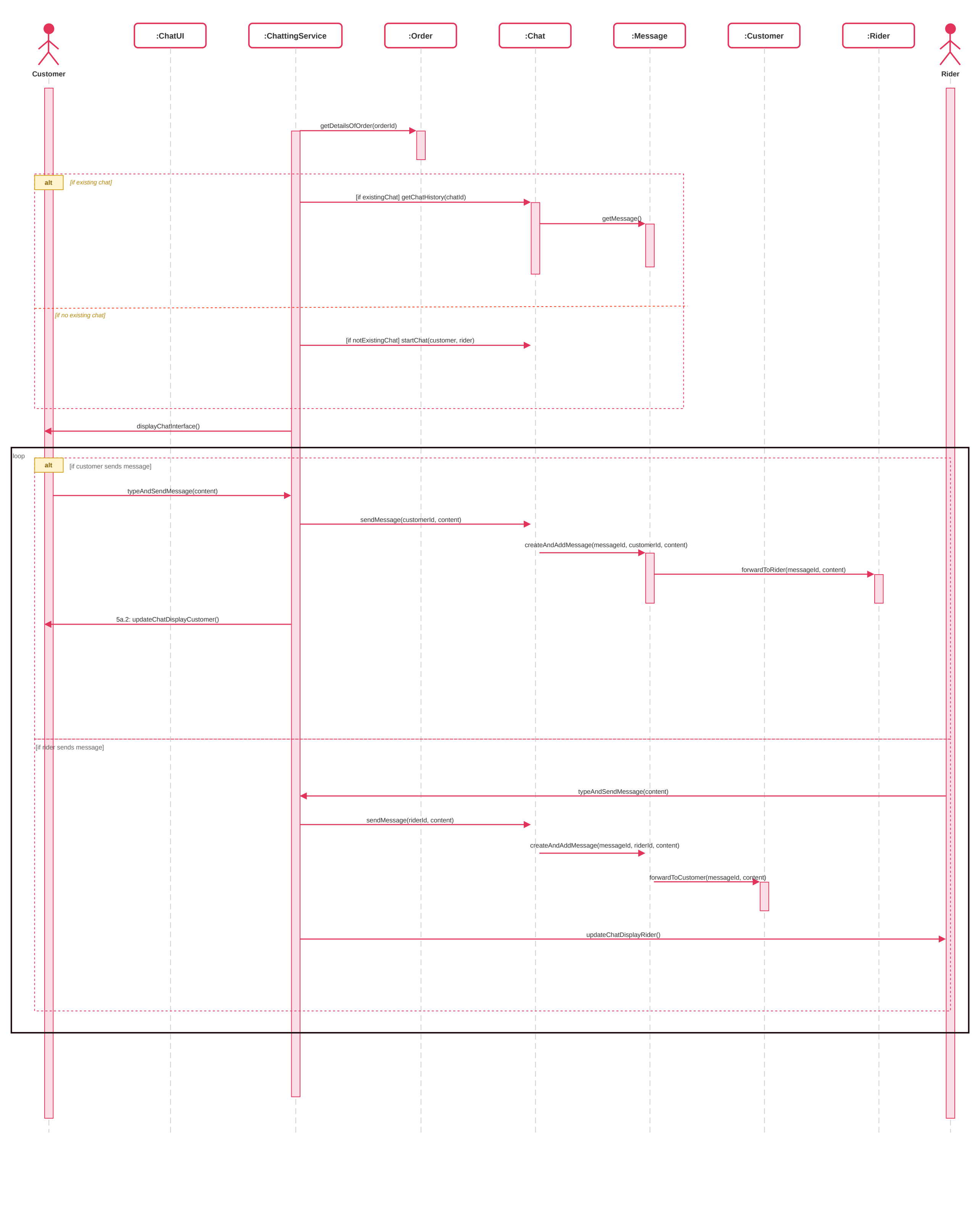
7. Rider Assignment and Dispatch


:RiderMatchingService
:Order
:Rider
:Timer
:Delivery
:Notification
getOrderDetails(orderId)
searchAvailableRiders(location)
sortRidersByProximity()
startTimer(15min)
loop
[for each available rider]
sendDeliveryOffer(orderDetails)
reviewOffer()
alt
[rider accepts]
acceptOffer(orderId)
createDelivery(orderId, riderId)
notifyRestaurant("Rider assigned")
notifyCustomer("Rider assigned")
[rider rejects]
rejectOffer(orderId)
checkTimer()
alt
[time ≤ 15 minutes]
timeRemaining
Loop continues
with next rider
findAlternativeRider()
[time > 15 minutes]
timeExpired
Loop exits,
order fails
notifyCustomer("No rider found")

8. Pickup and Delivery


:DeliveryHandler
:Order
:Location
:Notification
:Restaurant
:Customer
Rider
Phase 1: En Route to Restaurant
startDelivery(orderId)
getRestaurantLocation()
notifyRestaurant("Rider on the way")
arrivedAtRestaurant()
Phase 2: Pickup Food
updateStatus("Arrived")
confirmPickup()
updateStatus("Picked up")
notifyCustomer("Food picked up")
Phase 3: En Route to Customer
getCustomerLocation()
notifyCustomer("Rider on the way")
arrivedAtCustomer()
Phase 4: Complete Delivery
updateStatus("Arrived")
confirmDelivery()
updateStatus("Delivered")
notifyCustomer("Delivery complete")

9. Chat Between Rider and Customer


:ChatUI
:ChattingService
:Order
:Chat
:Message
:Customer
:Rider
Customer
Rider
getDetailsOfOrder(orderId)
alt
[if existing chat]
[if existingChat] getChatHistory(chatId)
getMessage()
[if no existing chat]
[if notExistingChat] startChat(customer, rider)
displayChatInterface()
loop
alt
[if customer sends message]
typeAndSendMessage(content)
sendMessage(customerId, content)
createAndAddMessage(messageId, customerId, content)
forwardToRider(messageId, content)
5a.2: updateChatDisplayCustomer()
[if rider sends message]
typeAndSendMessage(content)
sendMessage(riderId, content)
createAndAddMessage(messageId, riderId, content)
forwardToCustomer(messageId, content)
updateChatDisplayRider()